

2026 CCF 非专业级别软件能力认证第一轮模拟

(CSP-S1) 提高级 C++ 语言试题

模拟考试时间：120 分钟

考生注意事项：

- 试题纸共有 13 页，答题纸共有 1 页，满分 100 分。请在答题纸上作答，写在试题纸上的一律无效。
- 不得使用任何电子设备（如计算器、手机、电子词典等）或查阅任何书籍资料。

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1.

在 Linux 系统下本地测试函数式交互题。选手程序名为 `answer.cpp`，其中引用了头文件 `answer.h`，题目提供的交互库为 `grader.cpp`。希望生成可执行文件 `answer`，正确的编译命令是（ ）。

- A. `g++ grader.cpp answer.h -o answer -O2 -std=c++17 -static`
- B. `g++ grader.cpp answer.cpp -o answer -O2 -std=c++17 -static`
- C. `g++ answer.cpp -o answer -O2 -std=c++17 -static`
- D. `g++ answer.cpp answer.h -o grader.cpp -O2 -std=c++17 -static`

2.

关于最短路算法，下列说法中完全正确的是（ ）。

- A. 只要图中不存在负环，标准 Dijkstra 算法就一定能求出正确的最短路。
- B. Bellman-Ford 算法可在 $O(nm)$ 时间内求单源最短路，并能判断从源点可达的负环是否存在。
- C. Floyd 算法只能计算任意两点间的最短路，不能用于判断负环。
- D. 只要所有边权均不超过 2，就可以直接使用 01-BFS 求最短路。

3.

某算法的时间复杂度满足递归式

$$T(n)=4T(n^{1/2})+\Theta(\log^2 n)$$

则其渐进时间复杂度为（ ）。

- A. $\Theta(\log^2 n)$
- B. $\Theta(\log n \times \log \log n)$
- C. $\Theta(\log^3 n)$
- D. $\Theta(\log^2 n \times \log \log n)$

4.

使用数 1,2,3,4,6 各一个以及 2 个加号、2 个减号组成合法的后缀表达式，其值最大为（ ）。

- A. 10 B. 12 C. 14 D. 16

5.

运行如下 C++ 代码片段后，关于数组 `v` 的状态，下列说法正确的是（ ）。

```
vector<int> v = {10, 4, 7, 1, 9, 3, 8, 2, 6, 5};  
nth_element(v.begin(), v.begin() + 4, v.end());
```

- A. `v[4]` 必定为 5，且 `v[0]` 到 `v[3]` 必定依次为 1、2、3、4。
B. `v[4]` 必定为 5，且 `v[0]` 到 `v[3]` 包含 1、2、3、4。
C. `v[4]` 必定为 5，且整个数组已经完全有序。
D. `v[0]` 到 `v[4]` 包含 1 至 5，且右侧元素必定按从小到大排列。

6.

使用 `std::set<int> s` 维护含 `n` 个元素的有序集合。下列写法中，能在 $\Theta(\log n)$ 时间内找到第一个不小于 `x` 的元素的是（ ）。

- A. `s.lower_bound(x)`
B. `std::lower_bound(s.begin(), s.end(), x)`
C. 从 `s.begin()` 开始顺序枚举
D. 将元素复制到数组后进行冒泡排序再查找

7.

以下选项中，哪个序列不可能是某个长度为 7 的字符串的 KMP `next` 数组？规定 `next[i]` 为前缀 `S[1...i]` 的最长相等真前后缀长度。（ ）

- A. 0 0 1 2 0 1 2 B. 0 1 2 0 1 2 3
C. 0 0 1 1 2 3 0 D. 0 0 1 2 1 2 3

8.

对 12 个互不相同且均在 `[0,63]` 内的整数建立深度为 6（叶子到根的距离为 6）的 01-Trie。可能得到的最多结点数与最少结点数之差为（ ）。

- A. 21 B. 25 C. 28 D. 31

9.

用数据结构维护序列的区间运算结果。设二元运算“ $\odot$ ”满足结合律，但不保证满足交换律，也不保证每个元素存在逆元。下列说法正确的是（ ）。

- A. 一定可以只用前缀结果相减得到任意区间结果。
B. 普通树状数组一定可以支持单点修改和任意区间查询。
C. 线段树按固定的左右顺序合并，可以支持单点修改和区间查询。

D. 普通 ST 表利用两个可能重叠的区间，就一定能在 $\Theta(1)$ 时间回答询问。

10.

设 G 是一个含 12 个顶点的无向简单图， G' 为其补图。下列说法正确的是（ ）。

- A. 若 G 连通，则 G' 必然连通。
- B. 若 G 不连通，则 G' 必然连通。
- C. 存在 G ，使 G 与 G' 同时不连通。
- D. 若 G 是一棵树，则 G' 也一定是一棵树。

11.

设矩阵 $A = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$ 。

矩阵 A^{2027} 的每个元素对 5 取模后为（ ）。

- A. $\begin{bmatrix} 3 & 1 \\ 1 & 2 \end{bmatrix}$
- B. $\begin{bmatrix} 1 & 3 \\ 3 & 3 \end{bmatrix}$
- C. $\begin{bmatrix} 2 & 3 \\ 3 & 4 \end{bmatrix}$
- D. $\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$

12.

四维超立方体的 16 个顶点用长度为 4 的二进制串表示，两个顶点仅在恰有一位不同的时候连边，每条边长度均为 1。所有 $C(16, 2)$ 个无序点对间最短路长度之和为（ ）。

- A. 192
- B. 224
- C. 256
- D. 320

13.

有一排 9 盏灯，初始时全部关闭。每次操作选择一个非空连续区间，并反转区间内所有灯的状态。恰好进行 2 次操作后，这 9 盏灯可能呈现出多少种不同状态？（ ）

- A. 211
- B. 220
- C. 256
- D. 512

14.

长度为 1000 的排列初始为 $p_i = i$ 。现在恰好进行 4500 次操作，每次交换相邻两个位置上的数。操作完成后，新排列的最长上升子序列长度最短为（ ）。

- A. 99
- B. 100
- C. 101
- D. 111

15.

无向图 G 有 4 个顶点 A, B, C, D ，边集为

AB, BC, CD, DA, AC

将整数 1,2,3,4,5 互不重复地赋给这 5 条边。在全部 $5!$ 种赋权方案中，图 G 的最小生成树权值之和的总和为（ ）。

- A. 696
- B. 720
- C. 744
- D. 768

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填 $\sqrt{\quad}$ ，错误填 $\times$ ；除特殊说明外，判断题每题 1.5 分，单选题每题 3 分，共计 40 分）

(1)

```
01  #include <bits/stdc++.h>
02  using namespace std;
03
04  const int N = 19, mod = 1000000007;
05  int n, a[N][N], f[1 << N];
06
07  int main() {
08      cin >> n;
09      for (int i = 0; i < n; ++i)
10          for (int j = 0; j < n; ++j)
11              cin >> a[i][j];
12
13      f[0] = 1;
14      for (int s = 1; s < (1 << n); ++s)
15          for (int j = 0; j < n; ++j)
16              if (s >> j & 1)
17                  (f[s] += 1LL * f[s ^ (1 << j)]
18                   * a[__builtin_popcount(s) - 1][j] % mod) %= mod;
19
20      cout << f[(1 << n) - 1] << '\n';
21      return 0;
22  }
```

保证 $1 \leq n \leq 19$, $0 \leq a_{i,j} < 1000000007$, 完成下面的判断题和单选题。

● 判断题

16. (1 分) 当 $n=19$ 时, 程序不会访问数组 a 或 f 的范围之外。 ()
17. 若所有 $a_{i,j}$ 都是正整数, 则程序输出一定也是正整数。 ()
18. 删除转移式中的 $1LL *$, 对所有合法输入的输出均没有影响。 ()

● 单选题

19. 当输入为下列数据时, 程序输出为 ()。

```
3
1 2 1
2 1 3
3 2 1
```

- A. 34 B. 35 C. 36 D. 37

20. 该程序的时间复杂度为 ()。

- A. $\theta(n^2)$ B. $\theta(2^n)$
C. $\theta(n \times 2^n)$ D. $\theta(n^2 \times 2^n)$

21. 当 $n=13$ 时, 最内层 if 的条件为真的总次数是 ()。

- A. 53248 B. 49152 C. 106496 D. 114688

(2)

```
01 #include <bits/stdc++.h>
02 using namespace std;
03
04 using ll = long long;
05 ll len[90], dp[90][3];
06
07 void init() {
08     len[0] = 1;
09     len[1] = 2;
10     dp[1][1] = 1;
11     for (int i = 2; i <= 86; ++i) {
12         len[i] = len[i - 1] + len[i - 2];
13         int op = len[i - 1] % 3;
14         for (int p = 0; p < 3; ++p)
15             dp[i][p] = dp[i - 1][p]
16                 + dp[i - 2][(p - op + 3) % 3];
17     }
18 }
19
20 int solve1(int n, int p) {
21     string s = "0", nxt = "01", tmp;
22     while ((int)nxt.length() < n) {
23         tmp = nxt + s;
24         s = nxt;
25         nxt = tmp;
26     }
27     int ans = 0;
28     for (int i = p; i < n; i += 3)
29         ans += nxt[i] - '0';
30     return ans;
31 }
32
33 int value(int pos) {
34     if (pos <= 1) return pos;
35     int k = 0;
36     while (len[k + 1] <= pos) ++k;
37     return value(pos - len[k]);
38 }
39
40 int solve2(int n, int p) {
41     int ans = 0;
42     for (int i = p; i < n; i += 3)
43         ans += value(i);
44     return ans;
45 }
46
47 ll solve3(ll n, int p) {
48     int k = 0;
49     ll ans = 0;
50     while (len[k + 1] <= n) ++k;
51     for (; k >= 0; --k) {
52         if (len[k] > n) continue;
53         ans += dp[k][p];
54         n -= len[k];
55         p = (p - len[k] % 3 + 3) % 3;
56     }
57     return ans;
58 }
59
60 int main() {
61     ll n;
62     int p;
63     cin >> n >> p;
64     init();
65     if (n <= 1000000)
66         cout << solve1((int)n, p) << ' '
67             << solve2((int)n, p) << ' '
68             << solve3(n, p) << '\n';
69     else
70         cout << solve3(n, p) << '\n';
71     return 0;
}
```

72	}
----	---

假设 $1 \leq n \leq 10^{18}$, $p \in \{0,1,2\}$ 。程序统计斐波那契字符串前 n 个字符中, 下标对 3 取模等于 p 的位置上字符 1 的数量。

● 判断题

22. solve1 中各次字符串长度按斐波那契数增长, 因此构造字符串的总时间复杂度和额外空间复杂度均为 $\Theta(n)$ 。 ()

23. 将 init 中的 `int op = len[i - 1] % 3;` 改为 `int op = i % 3;`, 程序输出不变。 ()

24. 对所有 $n \leq 10^6$ 和合法的 p , 主程序输出的三个数总是相等。 ()

● 单选题

25. solve2 在最坏情况下的时间复杂度为 ()。

- A. $\Theta(n \times \log n)$
- B. $\Theta(n \times \log \log n)$
- C. $\Theta(n \times \log n \times \log \log n)$
- D. $\Theta(n \times \log^2 n)$

26. 对于输入 144 2, 输出结果的第一个数为 ()。

- A. 17
- B. 18
- C. 19
- D. 20

27. 若某次调用 solve3(n, p) 时, 语句 `ans += dp[k][p]` 恰好执行 7 次, 则 n 最小可能为 ()。

- A. 377
- B. 609
- C. 986
- D. 1596

(3)

01	#include <bits/stdc++.h>
02	using namespace std;
03	
04	using ll = long long;
05	using ull = unsigned long long;
06	const int N = 100005, M = 200005, S = 400037;
07	const int P = 1000000007, iv2 = (P + 1) / 2;
08	
09	int n, m, w[M], sum, dep[N], ct[M], sw[M], cur[M], ans[N];
10	ull h[N], he[M];
11	bool vis[N];
12	vector<pair<int, int>> G[N], T[N];
13	mt19937_64 rnd(random_device{}());
14	
15	int tot, hd[S];
16	struct node {
17	int nxt;
18	ull key;
19	} mp[M];
20	
21	int get(ull key) {
22	int u = key % S;
23	for (int i = hd[u]; i; i = mp[i].nxt)
24	if (mp[i].key == key) return i;
25	mp[++tot] = {hd[u], key};

```

26     hd[u] = tot;
27     return tot;
28 }
29
30 void dfs1(int u, int p) {
31     vis[u] = true;
32     for (auto e : G[u]) {
33         int v = e.first, i = e.second;
34         if (i == p) continue;
35         if (!vis[v]) {
36             dep[v] = dep[u] + 1;
37             T[u].push_back({v, i});
38             dfs1(v, i);
39             h[u] ^= h[v];
40             he[i] = h[v];
41         } else if (dep[v] < dep[u]) {
42             he[i] = rnd();
43             h[u] ^= he[i];
44             h[v] ^= he[i];
45         }
46     }
47 }
48
49 void dfs2(int u, ll c, ll b, ll b2) {
50     ans[u] = ((c + b * sum
51         - (b * b + b2) % P * iv2) % P + P) % P;
52     for (auto e : T[u]) {
53         int v = e.first, i = e.second;
54         ll x = w[i];
55         if (!he[i]) {
56             dfs2(v, c, (b + x) % P, (b2 + x * x) % P);
57             continue;
58         }
59         int j = get(he[i]);
60         if (ct[j] == 1) {
61             dfs2(v, c, b, b2);
62         } else {
63             int d = (x * (sw[j] - cur[j] * 2 - x) % P + P) % P;
64             cur[j] = (cur[j] + x) % P;
65             dfs2(v, (c + d) % P, b, b2);
66             cur[j] = (cur[j] - x + P) % P;
67         }
68     }
69 }
70
71 int main() {
72     ios::sync_with_stdio(false);
73     cin.tie(nullptr);
74     cin >> n >> m;
75     for (int i = 1, u, v; i <= m; ++i) {
76         cin >> u >> v >> w[i];
77         sum = (sum + w[i]) % P;
78         G[u].push_back({v, i});
79         G[v].push_back({u, i});
80     }
81     dfs1(1, 0);
82     for (int i = 1; i <= m; ++i) {
83         if (!he[i]) continue;
84         int j = get(he[i]);
85         ++ct[j];
86         sw[j] = (sw[j] + w[i]) % P;
87     }
88     dfs2(1, 0, 0, 0);
89     for (int i = 1; i <= n; ++i)
90         cout << ans[i] << ' ';
91     return 0;
92 }

```

假设 $3 \leq n \leq 10^5$, $2 \leq m \leq 2 \times 10^5$, $0 \leq w_i < 1000000007$, 输入图连通且无自环、无重边。不考虑异或哈希值相同造成的误判。

● 判断题

28. 函数 `get` 的最坏时间复杂度一定为 $\Theta(1)$ 。 ()
29. 对 `dfs1` 产生的一条树边 i , 若 `he[i] == 0`, 则在原图中删去边 i 会使图不连通。 ()
30. 若输入图是一条含 n 个顶点的链, 且每条边权均为 2, 则程序输出的所有数之和在模 P 意义下等于 $4n(n-1)(n-2)/3$ 。 ()

● 单选题

31. 这份程序对每个顶点 u 计算的是下列哪一项对 P 取模的结果? ()
- A. 选择两条边 $i < j$, 若存在一条从 1 到 u 的路径包含其中至少一条边, 则累加 $w_i w_j$ 。
- B. 选择两条边 $i < j$, 若存在一条从 1 到 u 的路径同时包含两条边, 则累加 $w_i w_j$ 。
- C. 选择两条边 $i < j$, 若每一条从 1 到 u 的路径都包含其中至少一条边, 则累加 $w_i w_j$ 。
- D. 选择两条边 $i < j$, 若每一条从 1 到 u 的路径都同时包含两条边, 则累加 $w_i w_j$ 。
32. 在 `dfs2` 中执行 `if (ct[j] == 1) dfs2(v, c, b, b2);` 时, 设当前沿树边 e 向下遍历。下列说法正确的是 ()。
- A. 删除 e 后原图的连通块数量一定增加。
- B. e 一定不属于任何简单环。
- C. 删除 e 后至少会出现两条新的割边。
- D. 删除 e 后不会产生新的割边。
33. 对于下列输入, 程序输出的第 8 个数为 ()。

```
10 10
1 2 12
2 3 18
3 4 24
4 5 36
5 6 8
6 7 10
7 8 12
8 9 14
9 10 16
10 5 18
```

- A. 10800 B. 11160 C. 11340 D. 12240

三、完善程序 (单选题, 每小题 3 分, 共计 30 分)

(1) (线性筛计算自幂)

给定正整数 n , 要求在线性时间内求出所有 $1 \leq i \leq n$ 的 i^i 对 1000000007 取模的结果。数据满足 $1 \leq n \leq 5 \times 10^6$ 。

在线性筛中同步计算 $f_i = i^i$ 。质数的值用快速幂求出; 合数第一次被筛到时, 把 f_{pq} 写成 f_p 与 f_q 的若干次幂之积。令 B 为 n 的平方根的整数部分, 将指数按 B 分块预处理, 并利用相邻质数之差增量维护另一部分幂。

试补全程序。

```

01 #include <bits/stdc++.h>
02 using namespace std;
03
04 const int N = 5000005;
05 const int SN = (int)sqrt(N) + 5;
06 const int mod = 1000000007;
07
08 int qpow(int a, int b) {
09     int res = 1;
10     while (b) {
11         if (b & 1) res = 1ULL * res * a % mod;
12         ①
13         b >>= 1;
14     }
15     return res;
16 }
17
18 int bsgs1[SN][SN], bsgs2[SN][SN];
19 bool vis[N];
20 int f[N], pr[N / 10], len;
21 int powers[N];
22
23 int main() {
24     int n;
25     cin >> n;
26     f[1] = 1;
27     const int B = sqrt(n);
28     for (int i = 2; i <= n; ++i) {
29         if (!vis[i]) {
30             pr[++len] = i;
31             f[i] = qpow(i, i);
32             if (i <= B) {
33                 bsgs1[i][0] = 1;
34                 for (int j = 1; j <= B; ++j)
35                     bsgs1[i][j] = 1ULL * bsgs1[i][j - 1] * f[i] % mod;
36                 bsgs2[i][0] = 1;
37                 for (int j = 1; j <= B; ++j)
38                     ②
39             }
40         }
41
42         powers[0] = 1;
43         int cur = 1, gap = 0;
44         for (int j = 1; j <= len && 1LL * i * pr[j] <= n; ++j) {
45             vis[i * pr[j]] = true;
46             int now = pr[j] - pr[j - 1];
47             ③
48             powers[ex] = 1ULL * powers[ex - 1] * f[i] % mod;
49             gap = max(gap, now);
50             cur = 1ULL * cur * powers[now] % mod;
51             ④
52             if (⑤) break;
53         }
54     }
55
56     for (int i = 1; i <= n; ++i)
57         cout << f[i] << ' ';
58     return 0;
59 }

```

34. ①处应填 ()。

- A. `a = 1ULL * a * b % mod;`
B. `res = 1ULL * a * a % mod;`
C. `a = 1ULL * a * a % mod;`
D. `a = 1ULL * res * res % mod;`

35. ②处应填 ()。

- A. $\text{bsgs2}[i][j] = 1\text{ULL} * \text{bsgs2}[i][j - 1] * \text{bsgs1}[i][B] \% \text{mod};$
 B. $\text{bsgs2}[i][j] = 1\text{ULL} * \text{bsgs1}[i][j - 1] * \text{qpow}(f[i], B) \% \text{mod};$

```
D. bsgs2[i][j] = 1ULL * bsgs1[i][j] * bsgs2[i][B - 1] % mod;
```

A. `for (int ex = 1; ex <= now; ++ex)`
 B. `for (int ex = gap + 1; ex <= now; ex += B)`
 C. `for (int ex = gap + 1; ex <= now; ++ex)`
 D. `for (int ex = 1; ex <= now; ex += B)`

A. $f[i * pr[j]] = 1ULL * bsgs2[pr[j]][i \% B] * bsgs1[pr[j]][i / B] \% mod * cur \% mod;$
 B. $f[i * pr[j]] = 1ULL * bsgs1[pr[j]][i \% B] * bsgs2[pr[j]][i / B] \% mod * cur \% mod;$
 C. $f[i * pr[j]] = 1ULL * bsgs1[i][pr[j] \% B] * bsgs2[i][pr[j] / B] \% mod * cur \% mod;$
 D. $f[i * pr[j]] = 1ULL * bsgs1[i * pr[j]][i \% B] * bsgs2[i * pr[j]][i / B] \% mod;$

A. `pr[j] > i`

B. `i < pr[j]`

C. `i % pr[j] == 1`

D. `i % pr[j] == 0`

给定一棵由主链和若干叶子组成的带权树。主链包含顶点 $1, 2, \dots, n$, 相邻主链顶点 $i, i+1$ 之间的边长为 a_i 。每个主链顶点还可以连接若干叶边, 所有叶边按输入顺序编号。共有 m 次操作:

数据满足 $2 \leq n \leq 8 \times 10^4$, $0 \leq k \leq 8 \times 10^4$, $1 \leq m \leq 8 \times 10^4$, 边长为不超过 5×10^8 的非负整数。

试补全程序。

```

01 #include <algorithm>
02 #include <iostream>
03 using namespace std;
04
05 const int N = 100005, M = 200005;
06 int n, k, m, z, tot;
07 int st[N], bel[N], id[N], lc[M], rc[M], fa[M];
08 long long a[N];
09
10 struct Node {
11     long long w, l, r, d;
12     int c;
13 } f[M], t[N << 2];
14
15 Node leaf(long long x) { return {0, x, x, x, 1}; }
16 Node path(long long x) { return {x, x, x, x, 2}; }

```

```

17
18 Node merge(Node a, Node b, char o, int s = 0) {
19     Node z = a;
20     if (o == 'R') {
21         if (a.c == 1) {
22             z.l = z.r = max(a.l, b.l);
23         } else if (s == 0) {
24             z.l = max(a.l, b.l);
25             z.r = max(a.r, a.w + b.l);
26         } else {
27             z.l = max(a.l, a.w + b.l);
28             z.r = max(a.r, b.l);
29         }
30         z.d = max(max(a.d, b.d), ①);
31     } else {
32         z.w = a.w + b.w;
33         z.l = max(a.l, a.w + b.l);
34         z.r = max(b.r, b.w + a.r);
35         z.d = max(max(a.d, b.d), a.r + b.l);
36         z.c = 2;
37     }
38     return z;
39 }
40
41 int join(int x, int y) {
42     int u = ++tot;
43     lc[u] = x;
44     rc[u] = y;
45     fa[x] = fa[y] = u;
46     f[u] = merge(f[x], f[y], 'R');
47     return u;
48 }
49
50 int build_rake(int l, int r) {
51     if (l == r) return l;
52     int mid = (l + r) >> 1;
53     return join(build_rake(l, mid), build_rake(mid + 1, r));
54 }
55
56 Node star(int x) {
57     return st[x] ? f[st[x]] : leaf(0);
58 }
59
60 Node make(int x) {
61     Node y = path(a[x]);
62     if (st[x]) ②;
63     return y;
64 }
65
66 void build(int x, int l, int r) {
67     if (l == r) {
68         t[x] = make(l);
69         return;
70     }
71     int mid = (l + r) >> 1;
72     build(x << 1, l, mid);
73     build(x << 1 | 1, mid + 1, r);
74     t[x] = merge(t[x << 1], t[x << 1 | 1], 'C');
75 }
76
77 void change(int x, int l, int r, int q) {
78     if (l == r) {
79         t[x] = make(l);
80         return;
81     }
82     int mid = (l + r) >> 1;
83     if (q <= mid) change(x << 1, l, mid, q);
84     else change(x << 1 | 1, mid + 1, r, q);
85     t[x] = merge(t[x << 1], t[x << 1 | 1], 'C');
86 }
87
88 Node query(int x, int l, int r, int ql, int qr) {
89     if (ql <= l && r <= qr) return t[x];
90     int mid = (l + r) >> 1;

```

```

091     if (qr <= mid) return query(x << 1, l, mid, ql, qr);
092     if (ql > mid) return query(x << 1 | 1, mid + 1, r, ql, qr);
093     Node p = query(x << 1, l, mid, ql, qr);
094     Node q = query(x << 1 | 1, mid + 1, r, ql, qr);
095     return ③;
096 }
097
098 void modify(int x, long long y) {
099     int u = id[x];
100     f[u] = leaf(y);
101     while (fa[u]) {
102         u = fa[u];
103         f[u] = merge(f[lc[u]], f[rc[u]], 'R');
104     }
105     if (④) change(1, 1, n - 1, bel[x]);
106 }
107
108 int main() {
109     cin >> n >> k >> m;
110     for (int i = 1; i < n; ++i) cin >> a[i];
111     for (int i = 1; i <= n; ++i) {
112         int c, l = tot + 1;
113         cin >> c;
114         while (c--) {
115             long long x;
116             cin >> x;
117             bel[++z] = i;
118             id[z] = ++tot;
119             f[tot] = leaf(x);
120         }
121         if (l <= tot) st[i] = build_rake(l, tot);
122     }
123
124     build(1, 1, n - 1);
125     while (m--) {
126         int o, x, y;
127         cin >> o >> x >> y;
128         if (o == 1) {
129             a[x] = y;
130             change(1, 1, n - 1, x);
131         } else if (o == 2) {
132             modify(x, y);
133         } else {
134             Node ans = ⑤;
135             if (x < y) ans = merge(ans, star(y), 'R', 1);
136             cout << ans.d << '\n';
137         }
138     }
139     return 0;
140 }

```

39. ①处应填 ()。

- A. (a.c == 1 || s == 0 ? a.l : a.r) + b.l
- B. (a.c == 1 || s == 1 ? a.l : a.r) + b.l
- C. (a.c == 1 ? a.d : a.w) + b.d
- D. (s == 0 ? a.w : a.l) + b.r

40. ②处应填 ()。

- A. y = merge(star(x), y, 'R', 0)
- B. y = merge(y, star(x), 'R', 0)
- C. y = merge(y, star(x), 'R', 1)
- D. y = merge(y, star(x), 'C')

41. ③处应填 ()。

A. `merge(q, p, 'C')`

B. `merge(p, q, 'R', 0)`

C. `merge(p, q, 'R', 1)`

D. `merge(p, q, 'C')`

42. ④处应填 ()。

A. `bel[x] < n`

B. `x < n`

C. `bel[x] <= n`

D. `st[bel[x]] == 0`

43. ⑤处应填 ()。

A. `x == y ? leaf(0) : query(1, 1, n - 1, x, y - 1)`

B. `x == y ? star(x) : query(1, 1, n - 1, x, y)`

C. `x == y ? star(x) : query(1, 1, n - 1, x, y - 1)`

D. `x == y ? star(x) : query(1, 1, n - 1, x + 1, y - 1)`